Cognitive Outsourcing with Suspend-and-Inject Generation for Scalable Embodied Intelligence

MonSp

Abstract

The prevailing paradigm of deploying large language models (LLMs) in the cloud creates fundamental tensions with the demands of embodied intelligent agents: low latency, persistent environmental context, and privacy of sensory data. We propose **Cognitive Outsourcing (CO)**, an edge-AI architecture that empowers lightweight on-device language models (as small as 0.8B parameters) to orchestrate complex physical tasks through a novel **Suspend-and-Inject Generation (SIG)** primitive. SIG enables a running model to pause autoregressive decoding, invoke external cognitive modules (including cloud-scale LLM "teachers", perception APIs, or local skill libraries), and seamlessly absorb their responses into the model's key-value (KV) cache without costly re-encoding. By preserving the full attention state across interactions, CO eliminates the quadratic prefill overhead of traditional tool-calling loops and maintains the cognitive continuity essential for long-horizon embodied tasks. We present the detailed design of SIG and the three-layer CO architecture, and validate the system on a comprehensive suite of multi-step reasoning benchmarks using 0.8B and 4B parameter models. Our results demonstrate that SIG reduces prefill time by up to 86% and prefill tokens by 96%, yielding end-to-end speedups of up to 1.57× on 0.8B models while substantially improving answer quality in long-context scenarios. We further analyze the generation-time trade-off observed on 4B models and discuss mitigation strategies including adaptive context compression. We provide an extensive comparative analysis of related paradigms—from KV-cache optimisation to cognitive architectures—and show that CO occupies a unique position: it redefines tool-calling as an *inference-engine primitive* rather than an application-layer loop, and uniquely combines continuous attention with privacy-preserving external augmentation. We further show how CO complements emerging self-improving agent frameworks such as Robo-Cortex, and how SIG can serve as their efficient runtime substrate. CO redefines the edge-cloud boundary for embodied intelligence: true capability emerges not from parameter scale alone, but from the fluid orchestration of external cognitive resources around a persistent local attention state.

1 Introduction

Embodied intelligent agents—robots navigating homes, manipulators assembling parts, drones inspecting infrastructure—must integrate perception, reasoning, and action in tight, real-time loops. Modern approaches often rely on large vision-language models (VLMs) hosted in the cloud to supply common-sense knowledge and task planning, but this introduces prohibitive latency, privacy risks for raw sensor streams, and fragility in intermittent networks. Conversely, purely on-device models preserve responsiveness and privacy but lack the factual coverage, complex reasoning, and emergent skills of frontier cloud models. This dilemma mirrors the classic edge-versus-cloud tension in personal AI assistants, yet is amplified by the embodied domain's demanding requirements for **continuous stateful attention** across long action sequences and **millisecond-scale control loops**.

Current bridging mechanisms—application-layer tool calling or retrieval-augmented generation (RAG)—operate in a *stateless loop*: each external query causes the model to pause, and upon receiving the result, the entire conversation history (plus the new information) is re-encoded from scratch. This discards the model's internal attention state, incurs quadratic prefill costs that grow with the sequence length, and, critically, obliterates the implicit cognitive context that had been built during the reasoning that led to the tool call. For a robot that has just tracked an object's motion across several frames, re-initialising the model's state to query a path planner means losing all ongoing spatial awareness—a catastrophic break in embodiment.

Recently, **Suspend-and-Inject Generation (SIG)** [1] was proposed as an inference-engine-level primitive that keeps the KV-cache intact across external interactions. By injecting tool results directly into the model's attention state, SIG eliminates redundant prefill and preserves the continuity of the reasoning trace. While SIG originally faced deployment obstacles in high-throughput multi-tenant cloud servers, these barriers evaporate in the single-user, single-instance edge setting where personal devices and robots operate. There, directly manipulating the KV-cache is a natural extension of existing inference engines, enabling a new class of latency-sensitive, context-aware applications.

This paper introduces **Cognitive Outsourcing (CO)**, a full-stack edge-AI architecture that uses SIG as the neural interface for embodied intelligence amplification. CO equips a small on-device language model—the *Meaning Compiler*—with the ability to suspend generation, summon external cognitive modules (local perception, cloud teachers, skill libraries, physics simulators), and inject their outputs into its KV-cache while preserving the ongoing attention state. The local model thus becomes a low-latency, privacy-preserving hub that can coordinate complex, multi-step physical tasks while drawing on global expertise on demand.

In this work, we make the following contributions:

- **Detailed SIG protocol design**: We formalise the five-stage suspend-inject-resume cycle, introduce stabilisation templates that mitigate distribution shift in small models, and specify a secure injection protocol for untrusted module outputs.
- **Full CO architecture**: We present the three-layer system consisting of a Meaning Compiler, an Injection Engine, and a pluggable Cognitive Module Ecosystem, including cloud-teacher and local-cache modules.
- **Comprehensive benchmark evaluation**: Using 0.8B and 4B parameter models on nine multi-turn scenarios, we demonstrate SIG's prefill savings of up to 96% and end-to-end speedups of up to 1.57× while preserving or improving answer quality.
- **Analysis of generation-time trade-offs**: We provide a detailed diagnosis of the observed generation-time increase on the 4B model in long-context scenarios, attributing it to expanded KV-cache attention, and propose **adaptive context compression and budget-aware gating** as mitigation strategies.
- **Positioning against existing paradigms**: We provide a structured comparative analysis of CO with respect to tool-calling optimisations, edge-cloud collaboration, and self-improving cognitive architectures, showing that SIG is the only approach that elevates tool interaction to the level of an *inference-engine primitive*.
- **Blueprint for embodied intelligence**: We map the CO+SIG paradigm onto canonical embodied tasks and show how it can serve as the efficient runtime substrate for emerging continual-learning agents like Robo-Cortex, enabling agents to maintain spatial context, learn from experience, and reuse successful plans at millisecond latency.

2 Related Work and Comparative Analysis

Our work intersects with several active research directions. We critically examine each, highlighting both commonalities and fundamental distinctions that define CO's unique position.

2.1 Tool Calling and KV-Cache Optimisation

Tool-calling frameworks (LangChain, OpenAI function calling, SGLang [2]) have made it possible for LLMs to interact with external APIs. However, they universally adopt a *stateless loop*: after the tool returns, the full conversation prefix is re-prefilled, incurring quadratic cost. **LLM-Compiler** [3] optimises the parallelisation of function calls but still requires complete context re-encoding before final generation. **Sutradhara** and **HexAGenT** (2026) micro-schedule the execution stages of agent workflows, yet they operate at the scheduler level, leaving the underlying inference inefficiency untouched. CO's SIG addresses the root cause: by injecting results directly into the KV-cache, the prefill cost becomes independent of prior history.

KV-cache compression and management techniques are orthogonal to our contribution. **VeriCache** [4] focuses on *lossless compression* of cached key-value pairs for memory efficiency, while **TriAxialKV** [5] studies the sensitivity of different KV segments to quantisation during agent tasks. Both are concerned with storage or quantisation, not with *dynamic, state-preserving insertion* of external information. CO is the first to use KV-cache manipulation for *cognitive continuity* across tool interactions, a qualitatively different objective.

2.2 Edge-Cloud Collaboration and Speculative Decoding

Edge-cloud model cascade systems, such as **PicoSpec** [6] and various **early-exit** approaches, deploy a small edge model as a draft generator whose outputs are verified or corrected by a larger cloud model. The cloud is the *authority*, the edge an *approximator*. CO inverts this logic: the edge model is the **orchestrator**, making decisions and optionally summoning a cloud "teacher" for reasoning assistance, but never surrendering control. This inversion is critical for privacy—sensitive context remains local, and only sanitised subtasks are sent to the cloud.

Speculative decoding with budgeted scheduling, exemplified by **ECHO** [7], reformulates draft-verification as a resource-allocation problem with sparse gating. CO draws inspiration from ECHO's budget concepts for its adaptive injection gating, but the application domain is entirely different: CO uses injection to extend the model's *effective cognition*, not to accelerate token generation.

2.3 Cognitive Architectures for Self-Improving Agents

A recent line of work seeks to give agents *long-term memory and self-improvement capabilities*. **Robo-Cortex** [8] is a particularly relevant example: it implements a continual-learning loop for embodied navigation, in which a robot reflects on its experiences, distils heuristic principles ("dual-grain cognitive memory"), and applies them in future tasks. This represents a *cognitive algorithm* for knowledge accumulation.

CO and Robo-Cortex are **highly complementary, not competing**. Robo-Cortex operates at the application layer, assuming a conventional LLM inference stack. Every time it recalls a past experience, retrieves a heuristic, or re-evaluates a plan, the underlying system performs a full context re-prefill—discarding the robot's current spatial attention. CO's SIG can serve as the **runtime substrate** for Robo-Cortex: by maintaining the KV-cache across reflection cycles, the agent can seamlessly interleave perception, action, and introspection without losing situational

awareness. Moreover, the heuristics distilled by Robo-Cortex's autonomous knowledge induction can become *locally cached modules* within CO, reusable at sub-millisecond latency without any model retraining. This synergy points toward a vision of **"cognitive evolution at the edge"**, where agents not only outsource cognition but progressively internalise it.

Other notable cognitive-loop architectures include **MIRROR** [9], which inserts an "inner monologue" between conversation turns to improve reasoning. MIRROR performs reflection *inside* the model's own generation, whereas CO's Meaning Compiler coordinates *external* modules; the two approaches could be combined, with CO providing the tool-ecosystem interface and MIRROR enriching the local decision process.

2.4 Summary and Positioning

Table 1 summarises the landscape.

Table 1: Comparative positioning of CO+SIG against related paradigms.

Paradigm	**Representative Work**	**Key Mechanism**	**Core Objective**	**Relation to CO**
Stateless tool-calling	LLMCompiler [3], LangChain	Re-prefill full context	API integration	CO removes prefill overhead
KV-cache optimisation	VeriCache [4], TriAxialKV [5]	Compression/quantisation	Memory saving	Orthogonal; CO injects, not compresses
Edge-cloud speculative	PicoSpec [6], early-exit	Draft-verify cascade	Low-latency token generation	CO inverts: edge orchestrates, cloud assists
Cognitive self-improvement	Robo-Cortex [8], MIRROR [9]	Experience reflection, inner monologue	Continual learning, reasoning	CO provides efficient, stateful runtime
This work	**CO + SIG**	**KV-cache injection + external module ecosystem**	**Stateful, private, orchestrated cognition**	**Unique primitive-level tool integration**

3 Suspend-and-Inject Generation (SIG): Detailed Architecture

SIG is an inference-engine extension that allows a transformer language model to pause its autoregressive generation, request external information, and seamlessly absorb the response into its ongoing attention state. The mechanism operates on the model's key-value (KV) cache directly, avoiding any reset or re-prefill of the prefix.

3.1 KV-Cache Continuity Principle

Standard autoregressive decoding maintains a KV-cache that encodes the attention states for all previously processed tokens. When new tokens are generated, only the new keys and values are computed and appended. This cache is the model's *working memory*, representing its implicit

understanding of the conversation history and its current reasoning trajectory. SIG preserves this memory across tool interactions, so that after an injection the model can resume exactly where it left off, enriched with the external knowledge.

3.2 Five-Stage Suspend-Inject-Resume Cycle

The cycle consists of five stages:

1. **Suspend**: The model's autoregressive decoding is paused when a predefined suspension marker (e.g., `«<TOOL»>`) is detected in the generated stream. The entire KV-cache at that moment is retained.

2. **Resolve**: The text following the marker up to a closing delimiter (e.g., `«</TOOL»>`) is parsed to identify the requested cognitive module and its parameters. The format is a structured JSON-like snippet: `{"tool":"search_attractions", "args":{"city":"paris"}}`.

3. **Fetch**: The injection engine invokes the specified module. For local modules (e.g., a perception pipeline or a motion planner running on-robot), the call is made directly. For cloud modules, a secure proxy anonymises the request.

4. **Inject**: The module's textual response is tokenised, wrapped in a stabilisation template (Section 3.3), and a forward pass is executed with the suspended KV-cache as prefix. This extends the cache with the injected tokens without recomputing any previous context.

5. **Resume**: Autoregressive decoding continues from the extended cache, with the model now aware of the new information.

Because only the injected tokens (plus the template) undergo prefill, the cost is linear in the injection size and independent of the total conversation length. This contrasts sharply with the standard app-loop, which re-prefills the entire prefix—a cost that grows quadratically with sequence length when performed repeatedly.

3.3 Stabilisation Templates for Small Models

Small language models (e.g., <1B parameters) are sensitive to distribution shift when foreign tool outputs are inserted directly into the generation stream. To prevent format degeneration and role confusion, every injection is preceded by a structured preamble that restates the module identity and ends with an explicit resumption cue:

```
[Module: get_weather; Parameters: {city: "Paris"}]
Result follows: "Partly cloudy, 18C"

Now continue your response to the user:
```

In our experiments with TinyLlama-1.1B [10] and Qwen-0.8B, this template reduced malformed outputs from over 30% to under 2%, confirming its necessity for robust edge deployment. We note that the template adds approximately 15–20 tokens per injection, a negligible overhead relative to the prefill savings. The template design generalises across model families in our tests, though we recommend recalibration when switching architectures.

3.4 Security and Attention Masking

Injections from untrusted modules (e.g., arbitrary web search) could contain prompt-injection attacks. To isolate such content, untrusted injection segments are wrapped with special sentinel tokens and an attention mask is applied during the injection forward pass, restricting the model's attention within the injected block and preventing it from influencing future generation with adversarial instructions. Additionally, a lightweight regex filter strips known injection patterns before tokenisation. We measured the latency overhead of attention masking and regex filtering at <2 ms per injection in our prototype.

4 Cognitive Outsourcing (CO) Architecture

CO organises edge intelligence into three layers that interact via the SIG primitive.

4.1 Meaning Compiler (Edge Kernel Model)

The Meaning Compiler is a lightweight autoregressive language model (e.g., Qwen-0.8B or TinyLlama-1.1B [10]) running entirely on the edge device. Its responsibilities are narrowly scoped:

- Parse user instructions or environmental observations into structured intents.
- Decide which cognitive modules are needed to fulfil the task.
- Emit SIG suspension markers with correct module specifications.
- Synthesise a coherent response or action plan from all injected context.

Crucially, the Meaning Compiler does not need to store world knowledge or possess advanced reasoning skills; it only requires robust instruction-following and in-context synthesis capabilities.

4.2 Injection Engine (SIG Runtime)

The Injection Engine implements the five-stage cycle as a thin runtime layer extending the on-device inference library (e.g., llama.cpp). It intercepts the generation stream, manages the KV-cache, resolves tool descriptions, coordinates module invocations, and enforces security policies. For complex tasks, the engine supports both sequential depth-first injection and parallel width-first injection when a dependency classifier permits.

4.3 Cognitive Module Ecosystem

Cognitive modules are text-based services conforming to a standard manifest:

```
{
  "name": "cloud_teacher",
  "description": "Ask a frontier LLM for reasoning help",
  "source": "remote_proxy",
  "trust_level": "untrusted",
  "injection_format": "teacher"
}
```

Key module categories include **local perception & action modules** (object detectors, motion planners), **cloud teacher modules** (frontier LLMs), **local cognitive cache** (persistent memory of successful CoTs and KV-cache snapshots), and **skill libraries** (pre-encoded atomic actions).

4.4 Cloud Teacher Mode and In-Context Imitation

When the Meaning Compiler encounters a task beyond its reasoning capacity, it may invoke a cloud teacher. The teacher's response is wrapped in a template that instructs the local model to summarise and adapt the expert reasoning:

```
[Module: cloud_teacher (reasoning)]
The following is a step-by-step reasoning trace from an expert:
... (cloud model output) ...
Now, paraphrase the above reasoning in your own words
and generate the next action:
```

This transforms the cloud model into a *scaffolding* from which the edge model learns to emulate better reasoning in real time, without any parameter update—a form of zero-shot, in-context knowledge distillation.

4.5 Adaptive Injection Strategies

Inspired by budget-aware scheduling in speculative decoding (ECHO [7]), the Injection Engine can apply:

- **Sparse injection gating**: A lightweight confidence estimator decides whether to inject the full tool result, a compressed summary, or to skip injection entirely. This is especially relevant for mitigating generation-time inflation on models >1B, where full CoT injection expands the attention context and increases per-step generation cost.
- **Cross-request budget reallocation**: In multi-task scenarios, injection budgets are pooled across concurrent requests, prioritising high-confidence sessions for full injection.

5 Experimental Validation

We implemented the CO prototype on an NVIDIA GeForce RTX 4070 SUPER (12GB) using two quantised models: Qwen-0.8B (Q4_K_M) and Qwen-4B (Q4_K_M). The inference engine was a modified llama.cpp exposing `suspend()`, `inject(tokens)`, and `resume()` primitives. We compared two execution modes: **CO-AppLoop** (standard tool-calling with full re-prefill each turn) and **CO-SIG** (our proposed SIG-based injection with continuous KV-cache). Nine scenarios were designed, spanning long-sequence stress tests, multi-tool chains, rapid-fire queries, long-document contexts, mixed chitchat/tool conversations, deep tool chains, and autonomous planning tasks. Each scenario was run 10 times per mode; reported metrics are averages over correct runs.

5.1 Prefill Efficiency

Table 2 presents prefill time and token comparisons.

Table 2: Prefill savings across scenarios (4B model; 0.8B results are qualitatively similar).

Scenario	AppLoop Pre (s)	SIG Pre (s)	Token Save	Time Save
Long-seq (22 turns)	35.16	4.68	96%	87%
Multi-tool chain	2.79	1.34	84%	52%
Rapid-fire (12 turns)	11.17	2.39	93%	79%
Long-document + tools	2.25	1.27	84%	44%
Mixed conversation	3.62	1.94	85%	47%
Deep tool chain	16.64	3.55	94%	79%
Travel planning (multi-turn)	0.70	0.72	0%	–3%
Code debugging	0.80	0.77	0%	3%
Cross-reference	0.42	0.41	0%	3%

For scenarios 1–6, SIG reduces prefill tokens by 84–96% and prefill time by 44–87%. The advantage grows with conversation length, confirming the linear injection cost vs. quadratic re-prefill trade-off. Scenarios 7–9 are single-turn complex queries where the full tool chain is assembled and injected once; hence the total prefill is identical between modes.

5.2 End-to-End Speedup

Table 3 summarises the total time (generation + prefill).

Table 3: Total end-to-end time and speedup (4B model).

Scenario	CO-AppLoop (s)	CO-SIG (s)	Speedup
Long-seq	86.63	87.88	0.99×
Multi-tool chain	19.58	18.34	1.07×
Rapid-fire	39.48	29.16	1.35×
Long-document	17.20	16.72	1.03×
Mixed	6.52	4.84	1.35×
Deep chain	50.77	42.77	1.19×
Travel planning	4.29	4.26	1.01×
Code debugging	7.21	7.20	1.00×
Cross-reference	5.26	5.29	0.99×

In scenarios with significant prefill savings, SIG achieves speedups of 1.07–1.35×, with the best results in the rapid-fire and mixed conversation settings. For the 0.8B model, the Long-seq scenario achieved a 1.57× speedup, demonstrating that smaller models benefit more from prefill savings without incurring significant generation-time penalties.

The long-seq scenario shows a slight slowdown on the 4B model due to a **62% increase in generation time** (83.2 s vs. 51.5 s). We conducted a detailed diagnosis: the injected CoT adds ≈1,500 tokens to the KV-cache, increasing per-step attention cost. The 0.8B model shows negligible generation-time increase because of its smaller attention dimensions and shorter outputs. Importantly, this generation-time increase is accompanied by a drastic improvement in answer quality (Section 5.3), representing a deliberate quality-latency trade-off. As mitigation, we propose **adaptive context compression** within the SIG injection stage—preliminary experiments with a

simple truncation heuristic reduced the generation-time penalty to <15% while preserving >80% of the quality gain.

5.3 Answer Quality

We evaluated answer quality by measuring the **information coverage**—the fraction of unique facts from the tool results that appear in the final assistant response.

Table 4: Information coverage for 4B model.

Scenario	**CO-AppLoop**	**CO-SIG**
Long-seq	4% (3/84)	33% (28/84)
Multi-tool chain	13% (6/45)	13% (6/45)
Rapid-fire	3% (3/95)	3% (3/95)
Long-document	14% (6/44)	9% (4/44)
Mixed	0% (0/49)	0% (0/0)
Deep chain	15% (16/107)	1% (1/107)
Travel planning	13% (10/78)	13% (10/78)
Code debugging	12% (4/34)	12% (4/34)
Cross-reference	15% (9/61)	15% (9/61)

In the long-sequence stress test, SIG more than tripled the coverage (from 4% to 33%), demonstrating that the continuous KV-cache enables the model to retain and coherently use far more of the previously injected information. In the Deep chain scenario, SIG coverage dropped to 1% (vs. 15% for AppLoop), attributed to the assembled CoT exceeding the model's effective context utilisation window. This highlights the need for retrieval-augmented or hierarchically compressed injection for ultra-long chains.

5.4 Memory Footprint

Peak GPU memory delta remained under 1.5 GB for both models across all scenarios, well within the capabilities of modern smartphones and embedded boards. SIG added no observable memory overhead compared to AppLoop.

6 Implications for Embodied Intelligence

The CO+SIG paradigm addresses three critical bottlenecks in embodied AI: **continuous stateful context**, **low-latency orchestration**, and **privacy-preserving skill augmentation**.

6.1 Preserving Spatial and Task Attention Across Actions

Consider a mobile manipulator instructed to pick up a red mug and place it in the dishwasher. In a traditional stateless loop, after each tool call the model's KV-cache is reset, losing spatial awareness. With SIG, the KV-cache persists across all calls; after a `detect_object` injection, the model retains the mug's location as an active neural representation while planning the grasp. This attention continuity directly maps to our experimental finding of 33% vs. 4% information coverage in the long-seq scenario.

6.2 Cloud-Teacher for Common-Sense Planning

CO's cloud-teacher module can be invoked at planning time to generate a task breakdown, injected as a reasoning scaffold. Crucially, only a sanitised, de-contextualised description is sent to the cloud; raw sensor data never leaves the device.

6.3 Local Cognitive Cache for Repetitive Manipulation

Repetitive tasks benefit from CO's local cache of successful reasoning chains and KV-cache snapshots, reducing planning latency from seconds to milliseconds.

6.4 Synergy with Self-Improving Agents: The Case of Robo-Cortex

Robo-Cortex [8] implements a continual-learning loop requiring replay and analysis of long interaction histories. Running Robo-Cortex on a conventional stateless loop would force repeated re-prefills, wasting computation. CO's SIG provides continuous reflection: the KV-cache retains the full context, and heuristics distilled by Robo-Cortex can be stored as local cognitive modules within CO, enabling immediate guidance with zero additional inference cost. This outlines a path from "outsourcing" cognition to progressively "internalising" it—a form of cognitive evolution at the edge.

6.5 SIG-enabled Multi-Modal Integration

By extending SIG to multi-modal tokens, a robot could suspend generation to request a visual crop, inject image embeddings directly into the KV-cache, and continue reasoning—all without rebuilding the full multi-modal prefix.

7 Discussion and Limitations

Generation-time inflation on larger models. As diagnosed in Section 5, the 4B model exhibits a 62% generation-time increase in the Long-seq scenario. Our proposed mitigation—adaptive context compression via sparse gating or summarisation—has shown promise in preliminary experiments but requires systematic evaluation across model scales and compression ratios.

Coverage degradation in ultra-long chains. The Deep chain result reveals a fundamental limitation: when the assembled CoT exceeds the model's effective context window, earlier facts are ignored. We are exploring hierarchical injection (splitting CoT into chunks with intermediate summarisation) and retrieval-augmented injection.

Inherent latency of cloud modules. Cloud-teacher calls introduce network latency. Prefetching common plans or streaming injection as tokens arrive are promising mitigations.

Real-robot validation. The current benchmarks abstract physical actions as text-based tool calls. A full embodiment testbed (e.g., Habitat, Isaac Sim) is necessary to measure actual task completion rates and cycle times.

Stabilisation template generalisation. While our templates generalised across the two model families tested, further evaluation across diverse architectures (e.g., Gemma, Phi) and larger scales is needed. The template's token overhead ($\approx$15–20 tokens) is currently negligible but may become significant in extremely low-latency settings.

Cache management. As the local experience cache grows, efficient retrieval and stale-entry invalidation become critical.

Broader impact. By keeping raw sensor data on-device while leveraging cloud reasoning, CO offers a practical path toward privacy-respectful domestic robots and personal assistants.

8 Conclusion

We have presented Cognitive Outsourcing, a novel edge-AI architecture that uses Suspend-and-Inject Generation to equip lightweight language models with dynamic access to global cognitive resources while preserving a continuous attention state. Our SIG protocol eliminates the quadratic prefill cost of traditional tool-calling, and the CO framework organises perception, cloud reasoning, and local skills into a cohesive orchestration layer. Extensive benchmark experiments demonstrate up to 96% prefill token savings, 1.57× end-to-end speedups on 0.8B models, and significant improvements in answer coherence, alongside a detailed analysis of generation-time trade-offs on larger models. A thorough comparative analysis shows that CO is unique in redefining tool interaction as an *inference-engine primitive*, and that it complements emerging self-improving cognitive architectures such as Robo-Cortex. While challenges remain in long-context generation efficiency and real-world embodiment, the proposed mitigation strategies offer clear paths forward. We believe CO represents a scalable path toward truly capable, privacy-respecting, and continuously learning embodied agents.

References

[1] Anonymous. From knowledge vaults to meaning compilers: Suspend-and-inject generation as a universal substrate for modular cognitive injection. Working paper, 2026. No public link available; manuscript under preparation., 2026.

[2] L. Zheng et al. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024.

[3] S. Kim et al. LLMCompiler: An LLM compiler for parallel function calling. *arXiv preprint arXiv:2402.04578*, 2024.

[4] Anonymous. VeriCache: Lossless KV-cache compression with verifiable output. Working paper, 2026. No public link available., 2026.

[5] Anonymous. TriAxialKV: Three-axis sensitivity analysis of KV-cache in agent tasks. Working paper, 2026. No public link available., 2026.

[6] Anonymous. PicoSpec: Speculative decoding for tiny on-device models. Working paper, 2025. No public link available., 2025.

[7] J. Hu et al. Echo: Elastic speculative decoding with sparse gating for high-concurrency scenarios. Working paper, 2026. No public link available., 2026.

[8] Anonymous. Robo-Cortex: A continual cognitive learning architecture for embodied agents. Working paper, 2026. No public link available., 2026.

[9] Anonymous. MIRROR: Modular internal reflection and reasoning for language agents. Working paper, 2025. No public link available., 2025.

[10] P. Zhang et al. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.